CAUSALLY SOUND MEMORY INVALIDATION FOR LONG-HORIZON SOFTWARE ENGINEERING AGENTS: TRANSITIVE CONSTRAINTS, PARTIAL OBSERVABILITY, AND VERIFICATION BUDGETS

Abdul Wasea
Independent Researcher
abdulwasea5678@gmail.com

ABSTRACT

Long-horizon coding agents depend on compressed or persistent memories of past tool outputs, repository observations, design constraints, and derived engineering decisions. When a repository changes, a remembered observation may remain historically true while no longer justifying an active decision. Existing retrieval, compression, and state-consistent memory systems address portions of this challenge, but reliable selective *transitive* invalidation under missing dependencies and bounded verification resources remains a sharply defined research target. We formulate engineering memory as a time-indexed, provenance-annotated justification hypergraph; distinguish observation validity, justification validity, and current action admissibility; and propose CAVI (Causal Assumption Verification and Invalidation), a research architecture combining change-impact analysis, dependency closure, uncertainty-aware verification, and guarded handoff. We specify conditional soundness guarantees for complete dependency graphs, illustrate unavoidable limits when dependencies are latent, and outline a reproducible benchmark with counterfactual repository mutations. The contribution here is a formal and experimental *research agenda*, not empirical evidence that CAVI outperforms prior systems.

1 Research question and contribution boundary

Research question

Given a repository transition $S_t \to S_{t+1}$, a history of agent observations, and previously accepted engineering decisions, how can an agent identify the *smallest safely sufficient* set of decisions requiring revalidation before taking its next action? The adjective "safely" is substantive: withholding a questionable decision may cost additional computation, whereas reusing an invalid decision can create silent defects.

Scope

The target is persistent coding-agent memory, especially tasks spanning multiple commits, agents, and context refreshes. We exclude ordinary semantic question answering as the primary success condition. A memory system may answer "What did we decide?" perfectly yet answer "May I still rely on that decision?" incorrectly. We therefore assess downstream code correctness, constraint violations, and review cost rather than recall accuracy alone.

Proposed contributions

1. A formal distinction among historical truth, current relevance, causal justification, and executable action permissions.

2. A dependency-and-provenance model in which individual decisions may require conjunctive or alternative sets of justifications.

3. An incremental invalidation procedure with clearly stated assumptions, fallbacks for missing edges, and bounded verification policies.

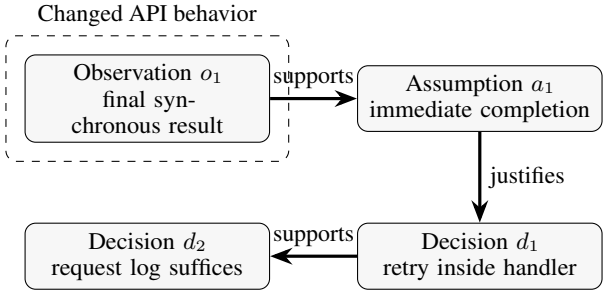
4. A controlled benchmark targeting transitive and hidden dependencies, with mutation-grounded oracles and cross-agent transfer.

5. A reproducibility and statistical-analysis protocol which can falsify, rather than merely celebrate, the proposed method.

Positioning. *MemTrace already provides provenance-aware, repository-state-aligned memory for coding agents. AMA-Agent already uses causal memory graphs. CAVI's proposed distinguishing test is selective revalidation of* derived engineering decisions *after upstream changes, especially when dependencies are incomplete; novelty and effectiveness are hypotheses, not settled facts [1, 2].*

2 Motivating failure: a payment service migration

Consider a payment service with the initial contract `POST /charge` returning an authoritative, final settlement status. An agent sees integration tests and adopts decision d_1: "Retry a failed charge synchronously within the HTTP handler." From d_1, it derives d_2: "The request log is sufficient for detecting duplicate charges." After a separate branch introduces asynchronous processing with delayed acknowledgements, both decisions may require review. An ordinary summary might still truthfully say that the endpoint historically returned a final status.



Three outcomes must be distinguished. **Over-invalidation** flags all past decisions, creating review cost. **Under-invalidation** fails to flag d_2, risking duplicate payments. **Correct selective invalidation** identifies the affected derivation paths and requires evidence that the affected decisions remain safe under the new contract.

The challenge is aggravated when a later agent never observes the original reasoning chain. Such an agent may hold only a compressed sentence, a repository snapshot hash, and a task plan. Recovering the dependency from those artifacts is a problem of partial observability, not merely token allocation.

3 Relationship to 2026 research and established systems

Method or line	Established contribution	Specific gap for this proposal
ACON (2026)	Optimizes context compression using failure-guided policies [3].	Does not establish complete transitive constraint invalidation.
AMA-Bench / AMA-Agent (2026)	Tests trajectory memory and causal-graph retrieval [2].	Answer correctness is not equivalent to mutation-triggered decision safety.
MemTrace (2026)	Immutable evidence traces, dependency organization, repository-aware validation [1].	Strongest close comparator; need isolate decision-level propagation and verification policy.
LangGraph	Checkpointed agent state and cross-thread stores [7].	Persistence does not automatically establish assumption validity.
Letta	Persistent agent-managed memory [8].	Durable storage still requires temporal and causal governance.
Incremental builds	Dependency-based selective recomputation [6].	Software-agent dependencies may be semantic, latent, and nondeterministic.
Truth maintenance	Retracts beliefs when supporting assumptions change [4].	Requires adaptation to repository observations and noisy LLM extraction.

The narrow conceptual advance

We treat a *memory entry* not as an atomic truth but as a claim bound to a proof obligation and a validity interval. A source-file hash may indicate that a file changed, but it cannot tell whether a policy or invariant that depended on that file changed meaningfully. Conversely, identical source files do not guarantee identical runtime contracts if dependencies, configuration, or external services change.

The closest theoretical connections are assumption-based truth-maintenance systems, dynamic dependency tracking, incremental view maintenance, software-change impact analysis, and causal inference. A rigorous paper must compare against these established ideas, not simply against embedding retrieval.

4 Formal model: repository and knowledge state

Let S_t denote the observable engineering state at time t, including repository content, dependency lockfiles, tested configuration, and accessible external contract versions. Let W_t denote the complete environment, of which S_t is an imperfect projection. The agent observes $O_t = h(W_t)$ and records events $e_t = (o_t, a_t, r_t)$ consisting of observations, actions, and tool results.

Definition 1 (Typed memory state)**.** *$M_t = (V_t, E_t, \pi_t, \sigma_t)$ is a directed justification hypergraph. Each vertex has type in $\{$observation, assumption, constraint, decision, plan, test$\}$. Hyperedges encode one or more premises needed to justify a conclusion. π_t maps claims to source evidence and timestamps. σ_t assigns statuses `active`, `suspect`, `invalid`, `superseded`, or `unknown`.*

A hyperedge $e = (P, v)$ states that all premises $P \subseteq V_t$ jointly justify v. Several hyperedges entering v represent alternative justifications. This avoids a naive graph error: if a decision is supported by two independent tests, invalidating one test does not necessarily invalidate the conclusion.

$$J_t(v) = \bigvee_{e \in \mathrm{In}(v)} \bigwedge_{u \in P_e} \mathrm{Valid}_t(u), \qquad \mathrm{Valid}_t(v) \not\equiv J_t(v). \tag{1}$$

The latter inequality emphasizes that justification validity is not sufficient for a decision's real-world correctness: the supporting observations may be incomplete or wrong. A sound proof about propagation within a complete graph does not provide a general guarantee that the graph captures every relevant dependency in the world.

5 Temporal semantics and supersession

Let a claim v have recorded interval $[t_b, t_e)$ and evidence provenance $\pi(v)$. Four temporally distinct predicates matter:

$$H(v, t_0) : \text{claim accurately described historical state at } t_0, \tag{2}$$
$$C(v, t) : \text{claim applies to current state at } t, \tag{3}$$
$$J(v, t) : \text{current premises justify the claim at } t, \tag{4}$$
$$A(v, t, q) : \text{using claim is safe for pending action } q. \tag{5}$$

For a system that changes continuously, $H(v, t_0) = 1$ and $C(v, t) = 0$ can both hold. A stale memory is not necessarily a false historical report. This distinction prevents two common mistakes: deleting audit-relevant evidence and treating preserved history as current authorization.

Supersession event

We model an update $u \rightsquigarrow v$ as a typed relation: `replaces`, `weakens`, `specializes`, or `contradicts`. A newer policy can supersede a prior policy without making its prior implementation incorrect for the earlier version. The system should preserve both, their validity windows, and the event authorizing the transition.

$$\mathrm{Usable}(v, t, q) = C(v, t) \wedge J(v, t) \wedge \mathrm{Authorized}(v, q) \wedge \neg \mathrm{Blocked}(v, t). \tag{6}$$

This formula is a desired *decision policy*, not a measurable ground-truth function in unconstrained environments. For evaluation, construct synthetic repositories and contracts with explicit oracles. For real repositories, use tests, specifications, maintainers, and a graded notion of confidence.

Temporal assertions require correct ordering across systems: commit time, observation time, ingestion time, and action time may differ. A delayed tool result must not silently supersede fresher evidence merely because it was ingested later.

6 The hidden dependency problem

Suppose a model writes "Use synchronous retries" but fails to record that this decision relies on finality of charge acknowledgements. The saved graph lacks the edge $a_1 \to d_1$. A graph traversal cannot invalidate what it cannot reach. This is the central methodological hazard.

Let $G^\star$ be the unknown complete graph of materially relevant dependencies, and $\widehat{G}$ the observed graph extracted from agent traces. Missing edges $E^\star \setminus \widehat{E}$ cause false negatives; spurious edges $\widehat{E} \setminus E^\star$ cause unnecessary verification. Different errors have different consequences.

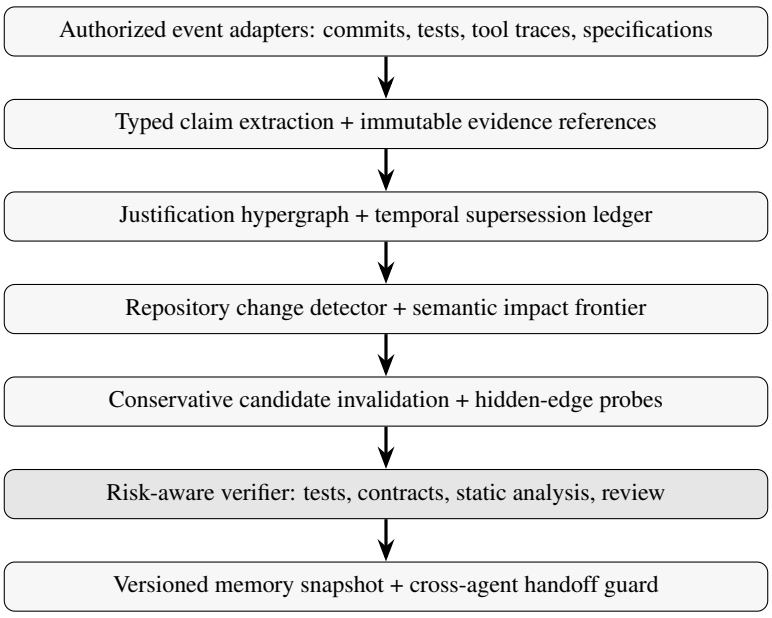
$$\mathrm{FNR}_{\mathrm{edge}} = \frac{|E^\star \setminus \widehat{E}|}{|E^\star|}, \qquad \mathrm{FDR}_{\mathrm{edge}} = \frac{|\widehat{E} \setminus E^\star|}{|\widehat{E}|}. \tag{7}$$

Candidate methods for discovering missing edges

- Static program analysis: call graphs, import graphs, dataflow, type dependencies, schema references.
- Dynamic observation: test coverage, tracing, runtime API contracts, observed state transitions.
- Structured elicitation: ask the authoring agent to list necessary premises at decision time.
- Counterfactual checks: perturb a premise in a sandbox and test whether the decision remains defensible.
- Human or tool adjudication for high-impact relationships that evidence cannot resolve.

The system should retain an `unknown-dependency` status. Confidence scores do not transform missing evidence into knowledge. Particularly risky decisions should be gated even when no explicit upstream edge was extracted.

7 CAVI architecture: proposed mechanism



Data separation

The immutable evidence ledger must not be overwritten by an LLM summary. The materialized memory view should be regenerated from evidence and justified claims. Repository files and tool outputs may contain untrusted instructions; these must be treated as data, not granted the authority to revise agent policy or bypass verification.

Interfaces

A practical interface may expose `record`, `on_change`, `suspect`, `verify`, and `handoff`. Each response should include a machine-readable status and evidence identifiers. When an action depends on unresolved high-severity claims, the integration can block, request approval, or supply a bounded warning depending on configured risk policy.

CAVI is intentionally proposed as a thin validation layer that can operate alongside established agent runtimes, not as a replacement for their persistence or execution engines.

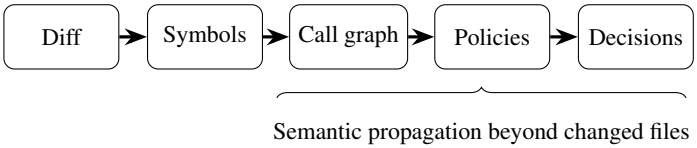
8 Change detection and semantic impact analysis

A commit diff $\Delta_t = S_{t+1} \ominus S_t$ identifies changed files, symbols, interfaces, configuration, tests, and dependency versions. A first-stage detector creates seed claims Z_t that explicitly depend on changed evidence. A second-stage program analysis expands the candidate impact frontier beyond textual overlap.

$$Z_t = \{v \in V_t : \text{evidence}(v) \cap \text{Touched}(\Delta_t) \neq \emptyset\}, \tag{8}$$

$$F_t = \text{Reach}_{\widehat{G}}(Z_t) \cup \text{RiskNeighbors}(Z_t). \tag{9}$$

The `RiskNeighbors` term permits conservative expansion when a dependency is likely but missing. Example signals include API call relationships, database migration references, and changed protocol schemas.



A narrow textual dependency heuristic has low recall when changes are indirect; a broad whole-repository review has high cost. The research question is to quantify this frontier-quality tradeoff, stratified by mutation type: interface contracts, security policies, configuration defaults, state-machine transitions, schema constraints, and dependency behavior.

Changed content alone is not a proof of invalidity: a comment-only edit should rarely force revalidation of payment invariants. Conversely, an external contract change might require revalidation even when local source code is byte-identical.

9 Incremental invalidation algorithm

Algorithm 1: Candidate invalidation with alternative support.

1. Ingest signed or otherwise attributable change evidence and compute directly affected premises Z_t.
2. Mark changed premises `suspect`; enqueue each downstream decision whose support set intersects Z_t.
3. For each queued vertex, re-evaluate each incoming justification hyperedge using current premise statuses.
4. If no certified justification remains, mark the vertex `suspect` or `invalid`, and enqueue its dependents.
5. If an unaffected alternative justification survives, retain provisional validity but record that a support path was lost.
6. Add risk-based candidates not reachable in the extracted graph; prioritize verification under the available budget.
7. Write a versioned verdict with evidence and unresolved assumptions; do not silently erase history.

$$\text{Support}(v) = \bigvee_{e \in \text{In}(v)} \bigwedge_{u \in P_e} x_u, \qquad x_u \in \{0, ?, 1\}, \tag{10}$$

where ? denotes incomplete knowledge. In a conservative three-valued policy, a conjunction containing ? cannot certify safety unless another fully verified support path exists.

Cost

With indexed edges, traversal over the impacted subgraph requires $O(|V_F| + |E_F|)$ graph operations, excluding verification and graph-extraction costs. Hyperedge evaluation adds a term proportional to the total sizes of visited premise sets. The heavy component is real-world verification, which may require tests, model calls, external contract inspection, or human review.

Important limitation. *A fast closure algorithm is not automatically causally sound. Its graph-completeness assumption must be made explicit; missing edges are handled through risk-sensitive conservative candidates, not magically discovered by traversal.*

10 Conditional correctness guarantees

Assumption 1 (Complete causal support for the evaluated domain)**.** *For every decision v, the stored hyperedges enumerate all material justifications, and every material state change is represented by a detected premise update. The verification oracle correctly adjudicates directly affected premises.*

Proposition 1 (Closure soundness within the formal model)**.** *Under complete causal support, correct direct-premise invalidation, deterministic monotone support evaluation, and a finite acyclic justification hypergraph, propagation to a fixed point flags each downstream decision that has lost all valid support paths.*

Proof sketch. Topologically order the acyclic justification graph. The claim is true for invalidated source premises by assumption. Suppose the status of all predecessors is computed correctly. The support formula is an OR of conjunctions, so a vertex has no certified support exactly when every justification includes an invalid or uncertified premise. Propagating changes and visiting impacted successors computes these statuses by induction. Unaffected vertices retain their previous certified support. □

What is not proved

The proposition does not certify semantic correctness of an LLM-generated dependency graph, completeness of tool observations, or safe execution in an open-world repository. Cycles require fixed-point semantics and potentially conservative treatment of self-support; nondeterministic tools can break the deterministic verification assumption.

$$\mathbb{P}(\text{missed unsafe decision}) \leq \mathbb{P}(\text{missing dependency}) + \mathbb{P}(\text{bad detector}) + \mathbb{P}(\text{bad oracle}) \tag{11}$$

The union bound is valid for a specified distribution of tasks, but each term must be measured or bounded; it is not a calibrated safety guarantee by itself. This analytical separation prevents unjustified "100% reliability" claims.

11 Risk-aware verification under a finite budget

At each step, candidate v_i has estimated probability p_i of unsafe reuse, loss magnitude L_i, verification expense c_i, and possible downstream risk interactions. Under a simple independent-benefit approximation, choose tests $x_i \in \{0, 1\}$:

$$\max_{\mathbf{x}} \quad \sum_i x_i \, p_i L_i \, q_i, \tag{12}$$

$$\text{subject to} \quad \sum_i c_i x_i \leq B, \tag{13}$$

where q_i is probability that the verifier detects the issue given an unsafe decision. This is a knapsack-like proxy, not a complete model: verifying one premise may make other checks redundant, and high-severity risks may need hard constraints independent of expected loss.

Beyond independence

For shared upstream premises, the benefit of a test is the risk reduction on the entire decision hypergraph, not simply its local score. One possible objective is $\max_{A:\text{cost}(A)\leq B} \mathbb{E}[L(\emptyset) - L(A)]$. The set function need not be submodular; greedy selection can be arbitrarily poor in some dependency structures. An experimental paper should therefore compare heuristic greedy, integer programming on small instances, random selection, and policy-driven priority classes.

$$\text{Priority}_i = \frac{p_i L_i q_i}{c_i + \epsilon}, \qquad \epsilon > 0. \tag{14}$$

This score is an implementation baseline, not a principled posterior estimate unless its components are calibrated. Severe data-loss, payment, and security constraints can be configured as mandatory gates, even when the estimated probability is low.

12 Probabilistic uncertainty propagation

An extracted edge may have uncertain existence. Let $z_{uv} \in \{0, 1\}$ indicate whether dependency $u \to v$ is real, and let $q_{uv} = \mathbb{P}(z_{uv} = 1 \mid \mathcal{D})$ be estimated from evidence $\mathcal{D}$. A Bayesian representation could update these beliefs as code changes, tests, and human annotations arrive.

$$\mathbb{P}(\text{Unsafe}(v) \mid \mathcal{D}) = \sum_{\mathbf{z}} \mathbb{P}(\text{Unsafe}(v) \mid \mathbf{z}, \mathcal{D}) \mathbb{P}(\mathbf{z} \mid \mathcal{D}). \tag{15}$$

Exact inference becomes intractable on large dependency graphs and requires assumptions about causal structure and observation generation. Monte Carlo sampling, variational approximations, or bounded conservative intervals are candidate methods. A more modest first implementation uses calibrated binary edge detection and preserves uncertainty explicitly.

Calibration experiment

Collect labeled dependency pairs in synthetic scenarios and human-audited real changes. Bin proposed probabilities and compare predicted to observed edge validity using expected calibration error and Brier score:

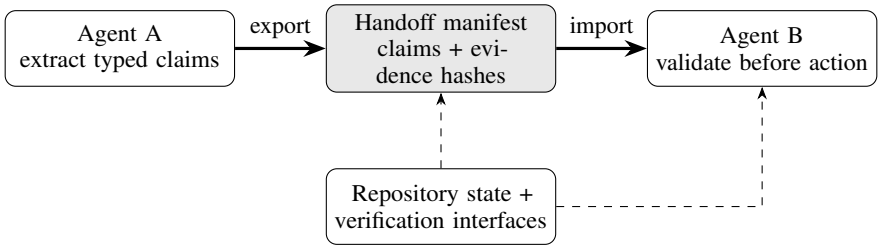
$$\text{BS} = \frac{1}{N} \sum_{i=1}^{N} (p_i - y_i)^2, \qquad y_i \in \{0, 1\}. \tag{16}$$

Graph topology and risk magnitude affect the meaning of calibration. A well-calibrated detector of direct edges can still underestimate downstream failure probability if the graph contains unobserved chains. Report calibration separately for direct, two-hop, and longer causal paths, and for different mutation classes.

When the system cannot distinguish stale and active decisions, the correct output is *uncertain and requiring inspection*, not a fabricated yes/no verdict.

13 Cross-agent handoffs and evidence contracts

A long-horizon workflow may move from Agent A to Agent B, or from one model family to another. Portable context must preserve *state and evidence*, not merely pblack. A proposed handoff manifest contains the active task, repository commit identifier, relevant constraints, currently trusted decisions, suspects, evidence hashes, unresolved questions, tool-permission boundary, and schema version.



Handoff invariant

A receiver must not treat imported claims as higher authority than its own system/developer instructions or verified repository state. Assertions from untrusted logs or repository text are data. Authorization to invoke tools, read secrets, or change files is never granted by the content of a memory entry.

$$\text{Accept}(m, S_{t+1}) \Rightarrow \text{SchemaValid}(m) \wedge \text{SourceBound}(m) \wedge \text{CriticalClaimsVerified}(m, S_{t+1}). \tag{17}$$

This is a normative acceptance rule. An empirical evaluation should vary the receiver model, serialization format, token budget, and freshness of the imported repository snapshot.

14 Benchmark construction: mutation-grounded oracles

We propose **TIDE-Bench** (Transitive Invalidation of Development Evidence), a benchmark name used here for a planned artifact, not an existing released dataset. Each episode starts from an executable repository and a known architectural contract. A sequence of tool observations and decisions is generated, followed by a controlled mutation whose causal effects are annotated.

Episode fields

- Baseline revision S_0 and executable tests identifying the intended invariant.
- Reasoning trace with observations, constraints, task decisions, and dependencies.
- One or more staged mutations $\Delta_1, \ldots, \Delta_T$.
- Gold set of invalidated premises and decisions, with rationales and reproducible test failures.
- Optional hidden dependencies, duplicate support paths, and distractor edits.
- A terminal coding task requiring correct use or rejection of remembered decisions.

Ground-truth quality

Oracle construction is difficult. Mutation operators should include schema reversals, API finality changes, authorization requirements, cache coherence, transaction boundaries, and dependency version semantics. For each episode, verify the mutated behavior through deterministic tests and independent review. Reject ambiguous cases; publish annotations and issue-tracking discussion for contested labels.

Partition by repository family, not just randomly by question, to prevent leakage of nearly identical dependency structures from train into test. Track complexity variables: hop distance, graph density, changed-symbol count, number of alternative justifications, and fraction of hidden edges.

15 Experiments, baselines, and ablations

Condition	Objective
Rolling natural-language summary	Establish weak but common compression baseline.
Full available history	Upper-budget comparator when history fits.
Vector retrieval over traces	Test semantic similarity without explicit dependency propagation.
LangGraph state + retrieval	Test strong persistence infrastructure with app-level retrieval.
MemTrace-style implementation	Strong repository-state-aware close comparator; faithfully implement published behavior where possible.
CAVI without semantic neighbors	Ablation: direct graph closure alone.
CAVI without alternative supports	Ablation: assess needless invalidation from naive edges.
CAVI without risk scheduling	Ablation: use random or FIFO verification.
CAVI full proposed method	Investigate additional value of uncertainty, provenance, and budget allocation.

Design

Use the same coding-agent backbone, tool permissions, repository snapshot, model temperature, and evaluation tests across all feasible conditions. Distinguish *memory accuracy* from *terminal code quality*. Repeat each stochastic condition with multiple random seeds and report per-repository and pooled statistics. Pre-register a primary endpoint such as critical-decision miss rate under a fixed verification budget.

When a reference method has no public compatible implementation, describe the reconstruction exactly and do not present it as an official reproduction. Add a human expert review baseline for a limited subset to anchor benchmark difficulty.

Negative controls

Insert comment-only changes and renamed local variables that leave semantics unchanged. A successful method should not flag every memory node; otherwise a trivial "invalidate everything" baseline could appear safest while being operationally unusable.

16 Measurement and statistical inference

Let $I^\star$ be the ground-truth invalidation set and $\widehat{I}$ the flagged set. Report precision, recall, and cost at fixed budget:

$$P = \frac{|I^\star \cap \widehat{I}|}{|\widehat{I}|}, \qquad R = \frac{|I^\star \cap \widehat{I}|}{|I^\star|}, \tag{18}$$

$$\mathrm{FNR} = 1 - R, \qquad \mathrm{ReviewCost} = \sum_{v \in \widehat{I}} c(v). \tag{19}$$

A safety-oriented paper should treat false-negative rate on high-severity decisions as a primary metric. Terminal patch correctness is independently measured by hidden tests and contract tests; do not infer correctness from self-reported agent confidence.

Statistical analysis

Use paired comparisons on the same episode and seed, cluster uncertainty by repository family, and report bootstrap 95% confidence intervals. For binary paired task success, McNemar's test or a hierarchical logistic model may be appropriate, depending on design and dependence structure. Report absolute differences, not just percentages, and avoid declaring superiority when intervals are wide.

$$\Delta_{A,B} = \frac{1}{N} \sum_{i=1}^{N} \Big(Y_i(A) - Y_i(B) \Big), \tag{20}$$

where Y_i is binary success or normalized loss for episode i. Report subgroup analysis by multi-hop depth, missing-edge rate, mutation severity, and handoff model pair. Where multiple outcomes are tested, clearly distinguish primary and exploratory hypotheses.

No real experiment has been run for this manuscript. *The figures that follow are synthetic demonstration data for presentation and analysis planning only.*

17 Chart I: synthetic invalidation tradeoff

The figure below illustrates how alternative methods might trace a precision–recall tradeoff as their candidate invalidation thresholds change. The coordinates were chosen solely to explain the evaluation plan and *do not* come from benchmarks or measured implementations.

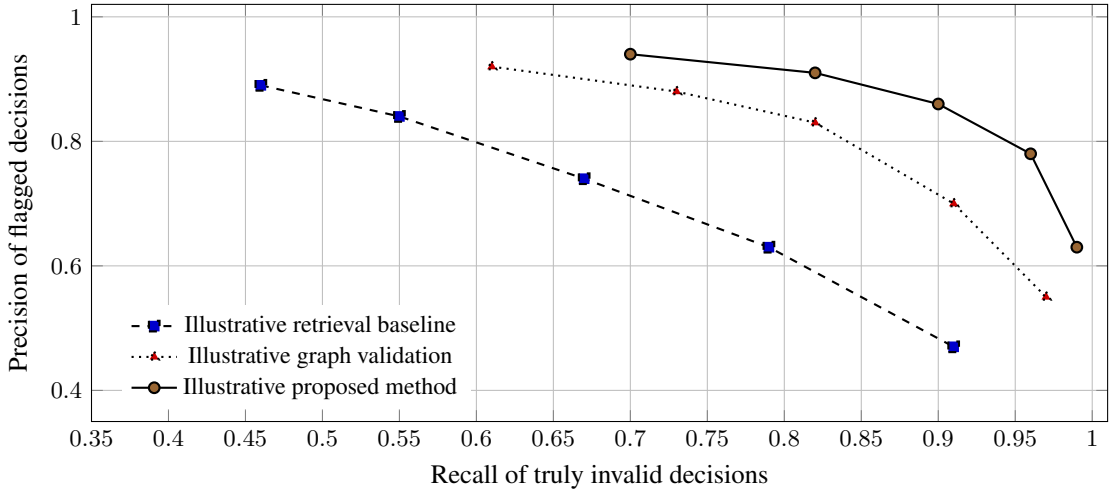


Figure interpretation. An attractive method would sustain high recall without triggering a large volume of false positives. High recall alone is not informative because blanket invalidation can trivially approach recall 1. Research experiments should report area under the precision–recall curve, cost-weighted recall, and critical-risk misses.

Ablations should establish where any actual benefit originates: observed dependency closure, hidden-edge search, alternative support handling, stronger tests, or more verification budget. Do not treat chart positioning as an expected quantitative claim.

18 Chart II: budget allocation and uncertainty

The next synthetic chart illustrates the expected qualitative structure of a verification-budget experiment. A larger budget permits more checks, but diminishing returns and noisy prioritization could leave some critical errors undetected.

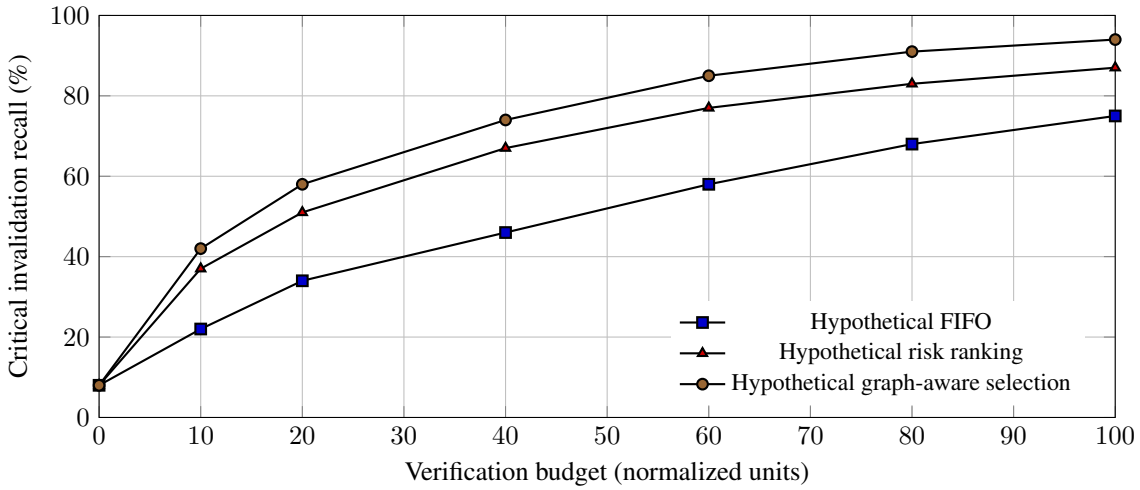


Figure interpretation. Plot actual recall against verified dollar cost, tokens, test runtime, and human-review minutes rather than only an arbitrary normalized budget. Separate the cost of constructing the dependency graph from the incremental cost of checking changed claims. A framework that spends hours building a graph to save seconds of future review may have poor end-to-end economics.

Additionally, analyze calibration under uncertainty: a decision with a low estimated failure probability and catastrophic loss may still warrant a mandatory test or human signoff.

19 Failure cases, security, and responsible deployment

Failure modes

- **Graph hallucination:** the extractor invents a causal edge, leading to expensive false invalidation.
- **Latent coupling:** a protocol or concurrency assumption influences a decision but is absent from saved traces.
- **Nonlocal environment drift:** an external dependency changes without a local diff.
- **False oracle confidence:** a test passes but does not cover the relevant invariant.
- **Circular reasoning:** a model cites its own prior summary as independent evidence.
- **Race conditions:** an agent verifies commit c_1 but acts on changed commit c_2.
- **Prompt injection:** attacker-controlled repository text attempts to elevate evidence into instruction authority.

Mitigations

Bind verification verdicts to exact repository and environment versions, and use optimistic concurrency checks before high-impact actions. Maintain separate policy authority and content evidence channels. Keep append-only audit records; support privacy-preserving retention policies and data minimization. Allow engineering teams to configure the severity of blocked decisions and human-approval requirements.

A safety benefit is not proven by demonstration of a single saved failure. Experimental evaluation should include adversarial mutations, contradictory tool outputs, stale snapshots, untrusted documents, and attempts to induce the system to bypass its guards.

Responsible claims

Avoid promising fully sound invalidation under an open-world, partially observed setting. More defensible claims concern measured reductions in unsafe reuse, accompanied by missed-case analysis and explicit assumptions about oracle completeness.

20 Research schedule, falsification, and conclusion

Practical research schedule

Stage	Deliverable
Weeks 1–2	Related-work matrix, typed memory schema, explicit causal semantics.
Weeks 3–4	Mutation generator and 20 manually reviewed pilot episodes.
Weeks 5–7	Incremental graph engine, test adapters, provenance logging.
Weeks 8–9	Comparable baselines, blinded evaluation, ablation harness.
Weeks 10–11	Expanded episodes, multi-seed runs, uncertainty analysis.
Week 12	Paper revision, artifact release, threat-to-validity audit.

Pre-registered failure criteria

The proposed approach would be weakened if (i) retrieval plus repository-aware validation matches its critical-decision recall at equal cost; (ii) hidden dependency detection yields too many false positives; (iii) graph construction costs dominate; (iv) results disappear under different model backbones; or (v) ground-truth labels cannot be made reproducible. These are meaningful reasons to reject the hypothesis, not engineering inconveniences to conceal.

Conclusion

The deeper technical problem is not memory persistence or compression in isolation. It is deciding which previously defensible engineering conclusions remain actionable when their evidence base changes. A promising approach combines explicit justification hypergraphs, time-indexed provenance, selective propagation, conservative treatment of missing dependencies, and verification calibrated to downstream risk. The key scientific challenge is to show that this machinery improves real coding outcomes rather than merely producing better-organized memory. The proposed CAVI architecture and TIDE-Bench protocol offer a falsifiable path to that result, while acknowledging substantial overlap with established truth-maintenance systems, incremental computation, and 2026 agent-memory research.

References

[1] Xu, H. et al. (2026). *MemTrace: State-Consistent Memory for Long-Horizon Coding Agents*. arXiv:2610.04838. https://arxiv.org/abs/2610.04838.

[2] Zhao, Y. et al. (2026). *AMA-Bench: Evaluating Long-Horizon Memory for Agentic Applications*. ICML, PMLR 306. https://proceedings.mlr.press/v306/zhao26bs.html.

[3] Kang, M. et al. (2026). *ACON: Optimizing Context Compression for Long-horizon LLM Agents*. ICML, PMLR 306. https://proceedings.mlr.press/v306/kang26b.html.

[4] Doyle, J. (1979). A Truth Maintenance System. *Artificial Intelligence*, 12(3), 231–272.

[5] de Kleer, J. (1986). An Assumption-Based TMS. *Artificial Intelligence*, 28(2), 127–162.

[6] Mokhov, A., Mitchell, N., and Peyton Jones, S. (2018). Build Systems à la Carte. *ICFP*. https://doi.org/10.1145/3236774.

[7] LangChain. *LangGraph Persistence Documentation*. https://docs.langchain.com/oss/python/langgraph/persistence. Accessed October 2026.

[8] Letta. *Memory Documentation*. https://docs.letta.com/guides/agents/memory. Accessed October 2026.

[9] Jimenez, C. E. et al. (2024). SWE-bench: Can Language Models Resolve Real-World GitHub Issues? *ICLR*. https://www.swebench.com/.

[10] Pearl, J. (2009). *Causality: Models, Reasoning, and Inference*, 2nd ed. Cambridge University Press.

[11] Pineau, J. et al. (2021). Improving Reproducibility in Machine Learning Research. *JMLR*, 22(164), 1–20.